Document number: P0592R4
Audience: EWG, LEWG

Ville Voutilainen
2019-11-25

To boldly suggest an overall plan for C++23

Many thanks to Herb Sutter, Bjarne Stroustrup, and Mike Miller for feedback on the early drafts of this proposal.

First things first

C++ ships on time.

C++11 shipped late, C++14 shipped on time, C++17 shipped on time, C++20 will ship on time, and C++23 will ship on time.

This plan doesn't change any of that. What is ready to ship can ship; what is not, will need to take the next train or a separate ship vehicle.

If a planned item cannot make it on time, we will postpone it; if it's in the Working Draft and deemed not ready, we will rip it out and ship the IS on time.

No, you don't need to get plenary approval to get a paper in the second or third priority bucket discussed

Before we go any further, the suggestion that material not explicitly discussed in this plan can't get air time is a hostile fairytale.

Just because executors, networking, reflection and pattern matching are high-priority items doesn't mean and will not mean that they will exhaust all time available. If we are realistic, we will admit to ourselves that while they are high-priority items, their priority doesn't prevent other material from being discussed; we will look at the proposals related to them, review them, and send said proposals to be revised. We are not going to just idly chat about them without doing anything else while the proposals are being revised.

Abstract

Various people have lamented our lack of direction, and that we don't have a plan for the next standard (or beyond). Since I haven't heard anyone promising to propose such a plan, here goes. In a nutshell, the plan is thus: for C++23, let's work towards having the following things in that standard:

- Library support for coroutines
- A modular standard library
- Executors
- Networking

Without a particular ship vehicle yet, we should also make progress on

- Reflection
- Pattern matching
- Contracts

All good? Agreed? Right, carry on. :) In case you want some elaboration, read on. I will elaborate on what C++23 Must Focus On and things that should progress during the standardization of C++23, but do not necessarily need to be in C++23.

High-level priority order, see here if you're concerned about issues and bug fixes

The priority order of handling material is thus:

1. Material that is mentioned in this plan.
2. Bug fixes, performance improvements, integration fixes for/between existing features, and issue processing.
3. Material that is not mentioned in this plan.

This means three things:

- Design groups will discuss material in that priority order.
- Design groups will, when sending material forward for wording review, designate which bucket the material belongs to.
- Wording review groups can use this priority order and said designation to prioritize their own work.

The priority order doesn't mean that bug fixes can't be considered before all on-plan material has been completely processed, or that off-plan papers can't be looked at while there are open issues. It does, however, allow wording-review groups to do issue processing even in a face-to-face meeting before looking at an off-plan paper, subject to the wording-review group's own prioritization.

Flushing the pipeline of features that we originally targeted for C++20 but couldn't get into C++20 is either at the boundary of (2) and (3), or just in (3), and that seems just fine.

C++23 must-work-on-first items

Any C++23-era WG21 subgroup meeting works on no other *NEW* material until all possible progress on these items is exhausted first

As stated already, "all possible progress" means at most a day or two of a face-to-face meeting, not all of a face-to-face meeting. Also note the emphasis on "new".

As stated in the abstract, these items are

- Complete C++20
  - Library support for coroutines
  - A modular standard library
- Executors
- Networking

Keen-eyed readers will notice that these are all library facilities, and the must-focus-on list contains no language facilities. That's intentional.

Library support for coroutines

Coroutines are in C++20. The only problem is that users can't do anything with them, because there are no types that are coroutine-aware. It should be fairly non-controversial that we need to fix this problem in C++23. What exactly are the coroutine-aware types that should ship in C++23 is to-be-defined by LEWG, but in general, the expectation would be that there are task types that would be plausible, but we should also take another hard look at which existing standard types could or should be made coroutine-aware. At any rate, the standard library should provide some coroutine-supporting types out of the box.

A modular standard library

Now that Modules are in C++20, C++23 should ship with a specification of how the functionality of the standard library is exposed as modules. While there are many arguably important libraries to modularize, the standard library is a very significant one, and should allow users to reap the benefits of modules.

Executors

We need executors for half-everything; we need them for networking, we probably need them for audio (if we go further in that direction), we need them for heterogeneous hardware, we need them for a better async, we need them for better task types. We have been working on executors for quite a while, and in C++23, we should deliver.

Networking

We should lower the barrier of writing networked C++ applications. We have a TS, we have years of existing practice, so once the blocker item is resolved, we should standardize networking. This shouldn't require much more rationale, but once the foundational facilities are in the standard library, writing cross-platform http code that requires no additional libraries to be built is right around the corner, and we should make that possible.

Items that should make progress, but can land later than C++23, or can land in a separate ship vehicle

Reflection

Reflection has a lot to offer: more powerful generic programming, including proxies, adapters, mediators; all sorts of things we use separate code generators for, persistence, remoting, glue code interfacing with other languages; aspect-oriented programming in general; better logging facilities, the list(s) go on and on.

Reflection (and code injection) covers a lot of ground, plugging feature holes in areas where programmers want and need to use C++, as opposed to trying to handle all this functionality with myriad smaller 'hard-coded' language extensions.

We have a TS, we have ongoing work for a constexpr-based approach, including prototype implementations. We should ship that work for the benefit of our user community, but when and in what form is not entirely clear yet.

Pattern matching

While we often say that C++ needs new control statements like it needs a hole in the head, pattern matching shows a lot of promise to deliver better filters/chains/selections than what we can approximate with pure-library facilities. Pattern matching improves type safety by making it easier to write type-safe code than writing the conventional C-style or traditional-C++-style alternatives. Also, it puts the visitor pattern out of business in many cases. It's not just another control structure; it interacts with the type system.

There's fair amounts of ongoing work in this area, and we should give it air time during the C++23 time frame. How to ship it, and in what form, is again not entirely clear yet.

Contracts

Redesigning Contracts is in early stages, and it's hard to foresee at this point what the pace of that work will be. It is, however, deemed reasonable to make progress in that area an on-plan item for this plan.

Where are nex-gen Ranges in this plan?

We could certainly entertain more Actions and Views in the realm of Ranges. Whether such material appears for standardization is a bit unknown at this point.

What about my favorite idea X, Y, foo and bar?

We need a plan on which we focus, to make sure that the important things that we need to deliver are delivered. In order to introduce new high-priority items into a plan like this, feel free to try and convince the committee that your favorite extension is

1. truly so important that it is either a must-work-on-first item or must-progress item
2. a good idea in general.

And again, if you have a proposal that is important, make sure it has a good rationale, and it will be discussed, either as an off-plan proposal (to which we will get during C++23 even if we have material with higher priorities) or as a priority 2 proposal.